

Provisional Application for Patent Cover Sheet

This is a request for filing a PROVISIONAL APPLICATION FOR PATENT under 37 CFR 1.53(c)

Inventor(s)

Inventor 1

Remove

Given Name

Middle Name

Family Name

City

State

Country

Brandon

Kindred

Cambridge

MA

US

All Inventors Must Be Listed – Additional Inventor Information blocks may be generated within this form by selecting the **Add** button.**Add****Title of Invention**

Automated Quality Assurance for Web Sites and Web Applications

Attorney Docket Number (if applicable)

QANA-001/00US 333031-2001

Correspondence Address

Direct all correspondence to (select one):

☒ The address corresponding to Customer Number☐ Firm or Individual Name

Customer Number

58249

The invention was made by an agency of the United States Government or under a contract with an agency of the United States Government.

☒ No.☐ Yes, the invention was made by an agency of the United States Government. The U.S. Government agency name is:☐ Yes, the invention was under a contract with an agency of the United States Government. The name of the U.S. Government agency and Government contract number are:

Entity Status**Applicant asserts small entity status under 37 CFR 1.27 or applicant certifies micro entity status under 37 CFR 1.29**

- ☒ Applicant asserts small entity status under 37 CFR 1.27
- ☐ Applicant certifies micro entity status under 37 CFR 1.29. Applicant must attach form PTO/SB/15A or B or equivalent.
- ☐ No

Warning

Petitioner/applicant is cautioned to avoid submitting personal information in documents filed in a patent application that may contribute to identity theft. Personal information such as social security numbers, bank account numbers, or credit card numbers (other than a check or credit card authorization form PTO-2038 submitted for payment purposes) is never required by the USPTO to support a petition or an application. If this type of personal information is included in documents submitted to the USPTO, petitioners/applicants should consider redacting such personal information from the documents before submitting them to USPTO. Petitioner/applicant is advised that the record of a patent application is available to the public after publication of the application (unless a non-publication request in compliance with 37 CFR 1.213(a) is made in the application) or issuance of a patent. Furthermore, the record from an abandoned application may also be available to the public if the application is referenced in a published application or an issued patent (see 37 CFR 1.14). Checks and credit card authorization forms PTO-2038 submitted for payment purposes are not retained in the application file and therefore are not publicly available.

Signature

Please see 37 CFR 1.4(d) for the form of the signature.

Signature	/Christopher Max Colice/			Date (YYYY-MM-DD)	2018-06-01
First Name	Christopher	Last Name	Colice	Registration Number (If appropriate)	65634

This collection of information is required by 37 CFR 1.51. The information is required to obtain or retain a benefit by the public which is to file (and by the USPTO to process) an application. Confidentiality is governed by 35 U.S.C. 122 and 37 CFR 1.11 and 1.14. This collection is estimated to take 8 hours to complete, including gathering, preparing, and submitting the completed application form to the USPTO. Time will vary depending upon the individual case. Any comments on the amount of time you require to complete this form and/or suggestions for reducing this burden, should be sent to the Chief Information Officer, U.S. Patent and Trademark Office, U.S. Department of Commerce, P.O. Box 1450, Alexandria, VA 22313-1450. DO NOT SEND FEES OR COMPLETED FORMS TO THIS ADDRESS. **This form can only be used when in conjunction with EFS-Web. If this form is mailed to the USPTO, it may cause delays in handling the provisional application.**

Privacy Act Statement

The Privacy Act of 1974 (P.L. 93-579) requires that you be given certain information in connection with your submission of the attached form related to a patent application or patent. Accordingly, pursuant to the requirements of the Act, please be advised that : (1) the general authority for the collection of this information is 35 U.S.C. 2(b)(2); (2) furnishing of the information solicited is voluntary; and (3) the principal purpose for which the information is used by the U.S. Patent and Trademark Office is to process and/or examine your submission related to a patent application or patent. If you do not furnish the requested information, the U.S. Patent and Trademark Office may not be able to process and/or examine your submission, which may result in termination of proceedings or abandonment of the application or expiration of the patent.

The information provided by you in this form will be subject to the following routine uses:

1. The information on this form will be treated confidentially to the extent allowed under the Freedom of Information Act (5 U.S.C. 552) and the Privacy Act (5 U.S.C. 552a). Records from this system of records may be disclosed to the Department of Justice to determine whether disclosure of these records is required by the Freedom of Information Act.
2. A record from this system of records may be disclosed, as a routine use, in the course of presenting evidence to a court, magistrate, or administrative tribunal, including disclosures to opposing counsel in the course of settlement negotiations.
3. A record in this system of records may be disclosed, as a routine use, to a Member of Congress submitting a request involving an individual, to whom the record pertains, when the individual has requested assistance from the Member with respect to the subject matter of the record.
4. A record in this system of records may be disclosed, as a routine use, to a contractor of the Agency having need for the information in order to perform a contract. Recipients of information shall be required to comply with the requirements of the Privacy Act of 1974, as amended, pursuant to 5 U.S.C. 552a(m).
5. A record related to an International Application filed under the Patent Cooperation Treaty in this system of records may be disclosed, as a routine use, to the International Bureau of the World Intellectual Property Organization, pursuant to the Patent Cooperation Treaty.
6. A record in this system of records may be disclosed, as a routine use, to a n other federal agency for purposes of National Security review (35 U.S.C. 181) and for review pursuant to the Atomic Energy Act (42 U.S.C. 218(c)).
7. A record from this system of records may be disclosed, as a routine use, to the Administrator, General Services, or his/her designee, during an inspection of records conducted by GSA as part of that agency's responsibility to recommend improvements in records management practices and programs, under authority of 44 U.S.C. 2904 and 2906. Such disclosure shall be made in accordance with the GSA regulations governing inspection of records for this purpose, and any other relevant (i.e., GSA or Commerce) directive. Such disclosure shall not be used to make determinations about individuals.
8. A record from this system of records may be disclosed, as a routine use, to the public after either publication of the application pursuant to 35 U.S.C. 122(b) or issuance of a patent pursuant to 35 U.S.C. 151. Further, a record may be disclosed, subject to the limitations of 37 CFR 1.14, as a routine use, to the public if the record was filed in an application which became abandoned or in which the proceedings were terminated and which application is referenced by either a published application, an application open to public inspection or an issued patent.
9. A record from this system of records may be disclosed, as a routine use, to a Federal, State, or local law enforcement agency, if the USPTO becomes aware of a violation or potential violation of law or regulation.

Automated Quality Assurance for Web Sites and Web Applications

Abstract

For companies producing web sites and web applications, being able to ensure the quality of their product is both financially burdensome and something that only the largest of corporations can properly afford. In order to ensure quality of their products, companies form *quality assurance* (QA) departments that are responsible for testing their products. While there are some processes that are considered good practice, quality assurance work is often regarded as more of an art than a science. As a result the efficacy of QA teams tends to decline as a product evolves. We set out to turn the art of QA into a rigorous engineering process that is simple, predictable, and capable of evolving with a project. Where there used to be loosely defined processes for creating tests, endless checklists, and armies of professionals responsible for following checklists, there is now a simpler process that takes a different approach to QA, producing automated working test cases in less time.

Introduction

The core function of the system is to generate User Interface (UI) tests whereby it would be the user's responsibility to verify the validity of the tests. The system then compares the result of running a test against these responses during subsequent runs. While this sounds simple, accomplishing such a process comes with a variety of complexities. It involved reimagining how to approach creating software quality assurance tests and solving multiple computational problems along the way. The biggest issue presented is the large number of combinations that are possible when generating tests. For example, if a web page has one thousand html elements, in order to evaluate which of these elements can be interacted with as a means of producing a test, it may be necessary to evaluate every possible mouse and keyboard interaction against every html element on the web page. Assuming that a website has multiple pages it is possible to end up with hundreds of millions of interactions that should be evaluated.

To mitigate this problem, a variety of known computer science techniques were utilized as well as a machine learning algorithm called reinforcement learning (RL). While implementing reinforcement learning, we encountered what is referred to as 'feature explosion' where the number of variables used to make a prediction grows uncontrollably. To solve this problem, we evaluated how the family of machine learning algorithms work and what changes could be made to make

calculations less computationally burdensome. In existing machine learning algorithms, a list of variables is created. These variables are used to learn what values between zero and one that each variable should have to make an accurate prediction. The downside of this process is that the variable list can't change in length or order, meaning that when the system makes a prediction it has to perform a computation for all variables regardless of if the value is helpful in making a prediction. In addressing this problem, we created a process that is capable of allowing the variable list to be of any size and in any order without changing the resulting prediction. The result of these modifications is that by only updating the subset of variables, calculating predictions becomes significantly less computationally intensive.

The Combinatorial Explosion Problem

The "feature explosion" problem alluded to above is known in computer science as the "combinatorial explosion problem": working through thousands of possible actions yields a problem space that has millions to trillions of combinations of actions that should be explored/tested. Unfortunately, this problem has non-deterministic polynomial (NP) hardness, which means that it can't be determined in polynomial time. To reduce the hardness of this problem, we use artificial intelligence (e.g., in the form of an artificial neural network running on a computer processor with weights, inputs, and outputs stored in non-volatile computer memory) to reduce the problem space to manageable number of combinations. Reducing the number of combinations reduces the amount of time required to test the web site or web application.

There are several techniques for reducing the size of the problem space. These techniques include, but are not limited to:

1. performing a bottom-up search (use graph theory to build out trees representing page states and actions, then explore each tree branch, starting at the leaf nodes and moving toward the root nodes; detect cyclical states and straighten out the graph that reduce complexity)
2. depth-first search in reverse; start at nodes and go toward root);
3. using partially observable Markov decision process to decide which actions should be performed (look at subset of information/partially observable space and reinforcement learning to converge rapidly on test action); and
4. distributing the work of testing combinations across multiple computer systems.

Automated Quality Assurance from a User Perspective

To use our automated quality assurance service, a user creates a domain, provides the domain name and the domain protocol (e.g., http, https, etc.) to the service, and then starts the discovery process. During discovery, different actors (bots) identify and explore the different interactive elements in the user's domain. Path exploration actors identify different elements (e.g., different links, forms, etc.) and predict changes in the domain's state (e.g., changes in page styling, hover states, transitions, changes to another page, etc.) based on possible user actions (e.g., mouse movements, mouse clicks, or keyboard strokes).

A work allocation bot allocates exploration work to exploratory bots based on the domain and the tasks. If the possible states and actions are represented as nodes and edges, respectively, of a graph, the work allocation bot identifies the leaf nodes, then assigns an exploratory bot to each leaf node. The exploratory bots, which may be tailored to particular elements (e.g., bots for testing forms that follow specific rules), emulate the actions for their nodes and note which actions result in state changes and which actions don't result in state changes. The bots, including the Url browser bot, the exploratory path browser bot, and the generic browser bot, compare the before and after states for each action.

Exploratory bots exploring different branches of the same tree defer to each other; as a first bot explores a branch, it labels nodes; a subsequent bot exploring the same branch sees the labels and realizes that branch has been explored already. In other words, the service uses cycle detection to identify states that have appeared before to avoid duplicating work.

The service creates tests based on steps/actions leading to state change(s), and the user verifies the outcomes of tests. The user can run the tests on demand and update them as the domain changes (e.g., if a link is updated, causing a test to fail, the user can update the test).

As the bots explore the domain, they continue to predict state changes in response to possible user actions. If the bots predict that an action is likely to result in a state change with a high enough confidence (e.g., $< 95\%$), they may perform the action; otherwise, they may stop and explore another possible action instead. In both cases, the bots feed the predictions and outcomes back into the system for reinforcement learning/backpropagation.

One difference between the system's reinforcement learning and conventional reinforcement learning is the use of "elastic feature vectors," described in greater detail below. In conventional machine learning, feature vectors must be a predetermined size before for the system to learn and make predictions. Conversely, our reinforcement learning process accepts feature vectors of varying lengths, which gives our system the ability to make decisions with partial knowledge. Succinctly, the system accepts data of arbitrary complexity containing words, numbers, lists, key-value pairs, tree structures, etc. It breaks down the data into small chunks of data referred to as a datum set. A datum set is used to generate a feature vector. The feature vector may vary in size based on the datum set. A datum set is used to retrieve the probabilities describing relationships between the datum set and the action set. (In this step, the system uses the raw datum set to find the probability vector, also referred to as a 'weight vector' in machine learning.)

The system constructs a probability array that includes the probabilities between the datum set and the action set, such that the order of the datum set represented in the vector matches the order of the datum set represented in the probability array. The process forks at this point for prediction and learning. For prediction, the system uses the datum vector and the probability array in a neural network to make a prediction. And for learning, the system accepts an action reward vector (a vector composed of reward values of actions based on performing a predicted action on the datum set). The system uses the datum vector, the probability array, and the action reward vector in a q-learning equation to generate updated probabilities for each datum-action relationship. The datum-action relationships can be updated through back-propagation

1. Terminology

Page state - A snapshot of what a page looks like based on the web elements visible within the page and other periphery data such as uniform resource locator (url) and screenshot.

Page Element - Data structure representing an html element as well as its attributes and styles.

Actions - Representation of an action that includes a label and a value. Actions, are limited to those that are able to be captured and controlled via web browser such as typing on a keyboard, using a mouse, capturing screenshots, and making http requests.

Path Object - Dynamic and flexible objects that can hold any sort of complex data as part of a node within a graph. This object exists to create flexibility by creating a sudo abstract data object that is similar to a node in a graph.

Path - A series of nodes in a graph. These nodes can be of any type, and together through their progression provide a map through a state space. Within Qanairy, we define 3 objects that make up a path (Page state, Page Element, Action).

Test - A test is an entity that validates that nothing changes when traversing a path through a state space. When traversing a path, the system asserts all page elements are visible to the user, that they haven't moved anywhere, and that they all have the same style for a given page state. The system then confirms that actions performed on a specified element result in a new page state that matches the path's next page state.

2. Distributed Artificial Intelligence using Actor architecture

a. What are actors?

Actors are independent bots that operate in parallel across a distributed network to accomplish a task.

b. Why distributed processing matters

When working with algorithms that result in near exponential growth of computational time, it becomes increasingly difficult to retrieve any meaningful result from a single machine regardless of size in any acceptable amount of time. In order to invert the algorithmic computational complexity, we utilize distributed actor architectures and narrowly defined chunks of work to distribute the work across servers, allowing new work to run in parallel. This results in the total run time shifting from quadratic time to logarithmic time.

c. Actors and their roles

i. URL Browser Interactor

Visits a given url and generates a page validation test, then passes the test to a Path Expansion actor.

ii. Browser Interactor

Crawls a site using a given path and captures the result. If the resulting page state is different than the last known page state in the path, then the Browser Interactor actor creates a test.

iii. Form Tester

Interprets rules on form fields and builds out tests for a form based on existing rules. This actor finds all forms on a page and loads the elements within each form. Once everything is loaded, the actor then investigates each element to see if there are rules in the system that apply. If there are, then the system assigns that rule to the element. After all elements have been examined, the actor then evaluates each of the rules on the various fields and builds all of the tests coded for within that rule's evaluate method. Once the actor has built a list of all possible paths based on the rules, the system then passes the data along so that a browser interactor can evaluate the path.

iv. Exploratory Path Browser Interactor

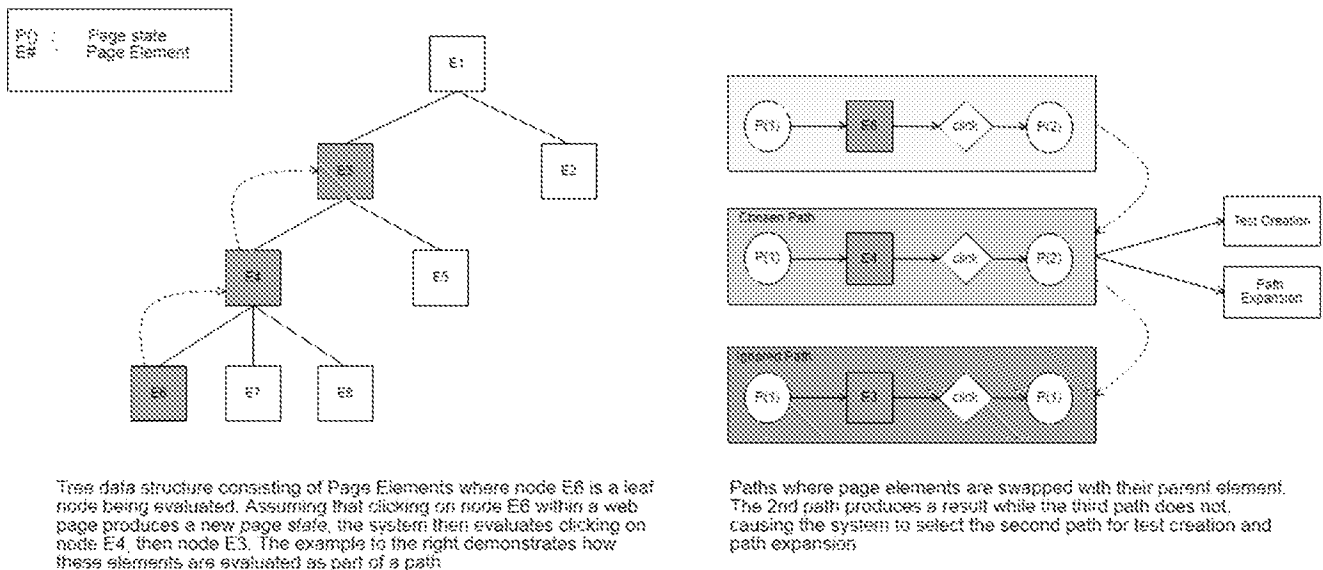
Expects to be provided with a message containing an exploratory path. If it does, then the actor crawls the path and attempts to get a favorable response by trying various actions that are provided as part of the exploratory path.

v.i Using interactions for real-time learning

The actions provided in the exploratory path are a map of data containing both the name of the actions and the probability that the actions would result in a favorable response, when performed against the last page element in the path. By performing these actions and recording the outcome, the actor determines if it should consider the action successful in producing any change in the last known state. The actor then labels the prediction as either correct (state changed) or incorrect (no state change), and sends it to the reinforcement learning engine to be used for learning

v.ii Reducing duplication with bottom up tree search

One downside to a system that relies on trying all potentially relevant interactions between html elements and actions is that the system is likely to identify a few elements that produce the same positive outcome. To remove this duplication, the system uses a bottom up tree search approach such that if the actor finds an action that produces a positive reward, it then creates a new path and replaces the original html element with its parent. If performing the action against the parent element results in a state change, the actor repeats the process, continuing to evaluate parent elements until interacting with a parent element doesn't result in a state change. Once the system identifies a parent element that does not produce a state change, then the system chooses the last known path to produce a state change to create a test and perform path expansion.



v. Work Allocator

Distributes work to the appropriate browser interaction actors (URL, Path, Exploratory Path).

vi. Path Expander

Identifies new permutations that extend an existing path by combining every visible web element on a page, with all possible actions.

vi.i Predicting action priorities for exploration

When creating test candidates, the system makes a prediction for which actions are most likely to produce a change in the page state. This prediction is then stored in the form of the action names and their associated predicted probabilities.

Vi.ii Reducing combinatorial complexity via active-pruning

Blindly searching every possible interaction within the state space would require an exponential amount of time in relation to the size of the state space. This brute force method would prevent any meaningful work from being done in a reasonable timeframe. To reduce the time that the system can fully search the state space, we use an active-pruning methodology to prevent the system from exploring paths that don't produce any changes to the state. For example if the bot were to click on an image element and nothing happened, then it would not continue pursuing other possible interactions beyond that. However, if the system were to click on a button and an error were to appear, the system would identify this as a valuable interaction and send the path to other actors for test creation and further exploration.

vii. Memory Registry Actor

Saves data to the database. The actor expects to receive an object and performs the steps to save it in the database. In some instances, this actor is also responsible for broadcasting the saved data to one or more users that have subscribed for real time updates.

viii. Testing Actor

Runs tests and reports the resulting statuses from running said tests to all users that are subscribed to receive updates for tests within a given domain.

Actor Diagram:

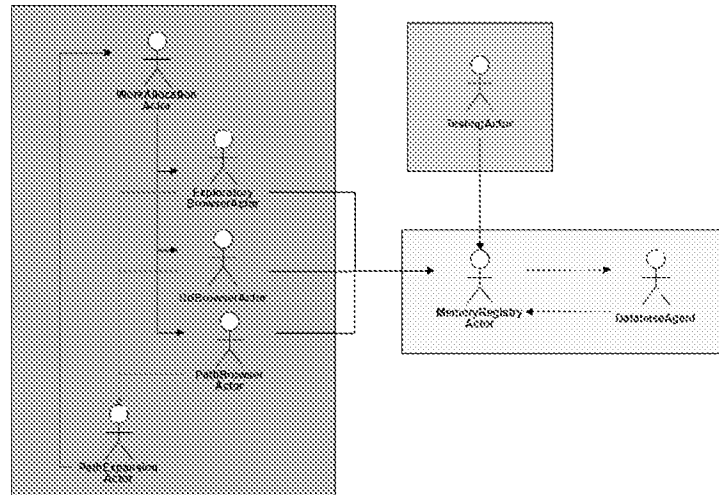


Figure 1: How data flows between various actors within the system

3. Path as a colored chain for data identification and processing

In order to make paths more dynamic within a strictly typed language, we defined a path as having some metadata and an ordered list composed of path objects. Path objects are dynamic and flexible objects that can hold any sort of complex data within the path, which really opens up the opportunities for what we can use paths for as well as what kind of data they can hold. There is a downside to this methodology, which is that the data in each path node becomes opaque and impossible to know what's inside without inspecting the data. This problem is best described as similar to what happens when you prepare for an Easter egg hunt using plastic eggs. After putting candy or other goods into each egg it becomes impossible to know what exactly is contained inside until you crack open the egg and look at the contents. To solve this problem, we attached a single piece of metadata to a path object when the path object is added to a path, which allows the system to identify the type of data held inside. We refer to this process of labeling path objects as coloring the nodes.

4. Rules engine and AI

What are rules and a rules engine?

Rules define what tests should be created for page elements. Rules are assigned to page elements by the system based on criteria defined within the *form tester actor*. The form tester actor also utilizes a rules engine, which is a system that allows a series of rules to be applied to various elements. The system can use these rules as an applied logic, useful in making decisions around generating new form tests.

Discovering and applying rules

Rules used by the system are coded in as having a label and a set of logic that can be evaluated against a *page element* as a means of generating a test path. In order to determine which rules should be applied to a page element, the system searches for html5 attributes that define how data should be validated. In the event that validation rules are applied elsewhere, such as via javascript, the system attempts to identify applicable rules as explained in the next section. When the system identifies a rule it checks that the rule hasn't been applied yet before adding it to a page element's list of rules

Evaluating presence of code based validations

Identifying input field length limits

Input fields can have both minimum and maximum lengths. To identify these, the system simultaneously creates a really long sequence of characters, and a single character. The system then proceeds to generate strings of characters by using linear descent to determine how long the next string should be until it finds a threshold in which the system either produces a value or doesn't. This threshold is defined as two values whose length differs by a single character in which one string produces an error and the other does not. Both of these would be coded into tests to validate the presence of string length validation rules.

Identifying numerical value limits

Input fields can have both minimum and maximum values. To identify these, the system simultaneously creates a really large number, and an equally large negative number. The system then proceeds to generate numerical values using linear descent until it finds both minimum and maximum value thresholds in which the system either produces an error or doesn't. This threshold is defined as two consecutive values in which one value produces an error and the other does not. Both of these would be coded into tests to validate the presence of numerical value limit validation rules.

Identifying character restrictions

The system is able to identify if an input field allows alphabetic, numeric, and special characters (e.g., @#\$\$%^&). The system identifies if these apply by creating separate tests where the field is filled in with single character values from each set of characters and monitoring for if the input results in an error

Identifying requirement rules

The system identifies if a field requires a value or not by attempting to submit a form without filling in a field. If an error occurs then the system evaluates if a requirement rule should be applied to a field.

Interpreting rules to build tests

In order to build tests, the system expects that each rule be coded with a specific method that is used to apply a rule to a test path and generate the possible outcomes. For example, a rule that a field requires input has a template coded for how to handle required fields and produces two paths. One path for when the field is not filled out and another for when it is.

4. Machine learning

a. Elastic Feature Vectors

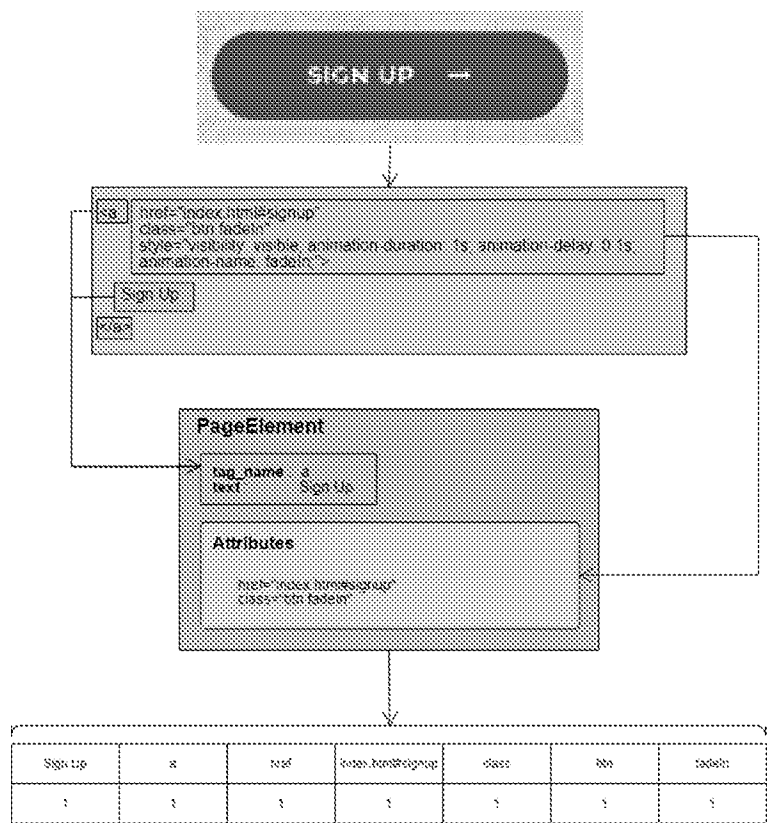
Feature extraction using hierarchical data decomposition

In order to process complex data structures in a way that allows it to be used in machine learning, we use a hierarchical method for reducing the data into small chunks that will be used as features. The process is simple, the system expects to receive a complex data structure encoded as a JSON object and then proceeds to iterate over the object's properties, cataloging each property and value into a separate data object. In the event that the property is also a complex object or data structure, the system then recursively calls this process until the object is fully mapped.

Data vectorization as a feature set

Once a complex data object has been broken down into small chunks (datums), its ready to be used in a machine learning algorithm. We

now take these datums and build out a feature vector with them. In order to do this, the system retrieves an ordered list of all datums that it has stored, which we refer to as a feature vector. The system loads the feature vector from the database and updates it with any data references that aren't in the feature vector. Once it is done making updates, the system makes a copy of the feature vector and begins populating it with the value one in all indices that match the datums from the complex data object decomposition. Once it is finished, the system has an elastic feature vector populated with zeros and ones and is now ready to be used with any machine learning algorithm.

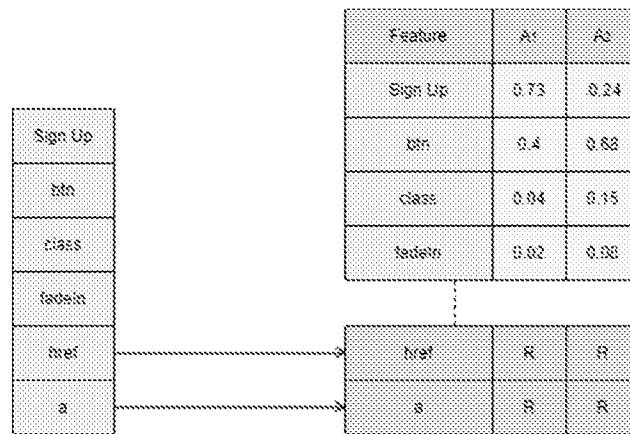


System retrieves html for a visible element, constructs a complex data representation, then vectorizes the data representation using hierarchical decomposition

Enabling data stability

In order for the concept of elastic feature vectors to work completely, the system should handle the unknown values associated with datums that were added to a feature vector. When the system updates a feature vector, a void is created, where the system is unsure of what

outcomes to expect when various actions are performed. Initializing the unknown provides proper stability to the data that will fully facilitate machine learning integration. In order to do this, when new datums are added to an elastic feature vector, they are also persisted as their own vertex. The system then retrieves all known action vertices and creates an edge between the datum and action vertices. The system then generates a random value between zero and one which it assigns as a weight to the edge. As a result, the system now has records that it can use as a basis for making predictions and learning.



A feature set is used to add unknown features into the existing weight matrix for feature action pairs by initializing the unknown value to a random number(R) between 0 and 1.

b. Making predictions

Getting data into Artificial Neural Network

With elastic feature vectors in place, loading data for use in an Artificial Neural network or any other machine learning algorithm is a breeze. The system loads the elastic feature vector as described in the previous section on *Elastic Feature Vectors*. Once it has loaded the feature vector, the system then retrieves all weights between features and actions which are then loaded into a matrix called a weight matrix. This matrix holds the best-known probabilities that each action will produce a favorable result when performed on a given set of features. This weight matrix is used for a multitude of machine learning algorithms including Artificial Neural Networks, and the system can now attempt to make predictions.

Reducing computational time by reducing elastic vector size

In most machine learning algorithms, a missing variable is coded as zero. This is done because it is important to make sure that the vector length matches the weight matrix length. Since we have a way of coding for ordered values through elastic feature vectors, we can opt to only load action weights for features that are active (having a value of one). By doing this we reduce the average load time of the algorithm by bounding the problem to the features present. We also reduce the computational time by removing unnecessary matrix computations that are guaranteed to equal zero.

Calculating a prediction using feature vectors

Using elastic feature vectors, we are guaranteed to have a set of features with activation values that are greater than or less than zero. This is then used along with the weight matrix to generate a prediction using a neural network.

Favoring prioritization over activation

By their nature, activation functions are highly restrictive on what is predicted, and if the algorithm predicts incorrectly then the entire process of prediction should be tried again after changes to the data are made. This process is not only computationally intensive but it also has longer learning cycles. Instead of this, we use the predicted probabilities to prioritize which actions should be explored. Once an action is explored the weight matrix is updated using the learning method described in *Learning feature policies*

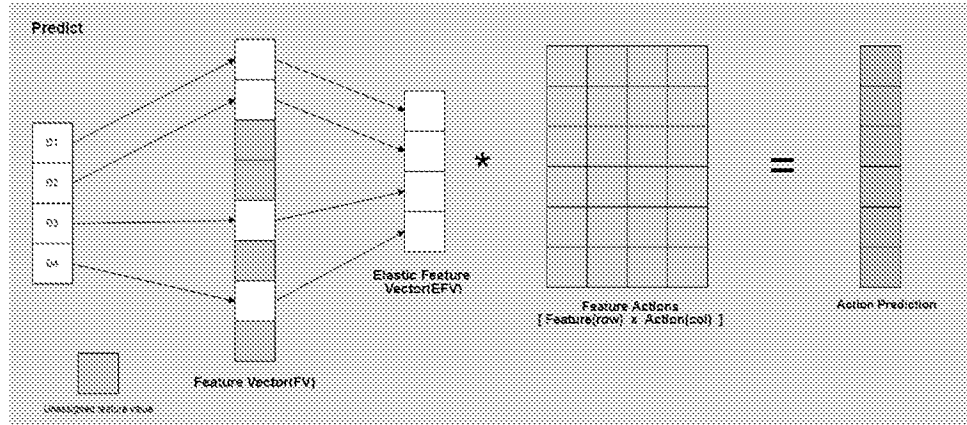


Figure 2: Process and matrix math used to predict how actions should be priorities for a set of features

c. Policies

A set of values between zero and one that represent probabilities that are used to determine which action should be executed based on a given state. Within our system, we generate a policy from a set of smaller policies that we call *feature policies*. Each policy is dynamically generated by loading all relevant feature policies for a state space and combining them into a matrix structure.

What is a policy?

A policy is stochastic and defines the order in which actions should be performed when searching for a productive action. Policies are made up of smaller policies referred to as feature policies. Feature policies are defined as a set of probabilities that correspond to a set of possible actions such that an action will produce a favorable result for any given feature. These feature policies are combined to form a state space policy that is defined dynamically by the features extracted through decomposition of complex objects. Creating this distinction allows us to create highly dynamic policy spaces that are capable of growing in complexity. Since the policy is not constrained to a specific size or arrangement we are free to add in layers representing data combinations of any variety. This means that the system is not only capable of learning for individual datums, it is also capable of learning how complex combinations of data affect outcomes.

d. Persisting policies in knowledge graphs

What is a knowledge graph?

A knowledge graph is a set of vertices that are connected via weighted edges, creating a graph that encodes how each piece of information relates to all other information. The weights assigned to edges between data vertices and action vertices represent feature policy values. By storing the data in this way it is easy to construct new policies on the fly by simply using a query to request all relevant data vertices and their action edges from the database and loading them into a feature action matrix.

By storing policies in a knowledge graph we are able to generate complex and ever expanding policy spaces, allowing the system to quickly and dynamically produce predictions for unfamiliar feature sets using the existing knowledge it has acquired. As a result the complexity of learning a policy for any given state space is much simpler.

Reducing query size for faster load times.

In most machine learning algorithms, the weight matrix is multiplied by the feature vector to make a prediction. In doing this, for each feature that is set to zero the resulting value in the equation will also be zero. This means that these values don't actually need to be loaded or even present for the machine learning process to still work. By not loading the data for the features that are set to zero in a given feature vector, we can reduce the computational burden of reading the features from memory and it also removes the possibility of there being too many features to either be retrieved or calculated quickly. Loading and computing against only what is relevant allows the system to guarantee rapid memory retrieval and linear computation time based on feature subsets.

e. Learning

Skipping the coin toss and learning multiple things at once

In traditional reinforcement learning systems, the system would drive knowledge by either making a prediction or selecting a random action. The downside of this is that the system learns slower because fewer records are created at a time. In order to acquire the desired records without introducing a bottleneck around choosing just one action,

the system prioritizes actions and tries them one at a time until a positive reward is experienced or there are no more actions to try. This allows the system to produce multiple records at once and significantly reduce the time it takes for the algorithm to learn to identify a clear winning action choice.

Policy learning calculations

To learn how a policy should change we utilize the Q-learning equation which is defined as follows:

$$Q(s_t, a_t) \leftarrow \underbrace{(1 - \alpha)}_{\text{old value}} \cdot \underbrace{Q(s_t, a_t)}_{\text{add value}} + \underbrace{\alpha}_{\text{learning rate}} \cdot \left(\underbrace{r_t}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(s_{t+1}, a)}_{\text{estimate of optimal future value}} \right)$$

learned value

This can be translated to the following diagram that represents this as a matrix calculation. The following diagram is showing the process for a single feature policy. The size and dimensions of the matrices are likely to change based on actions or features in a given state space.

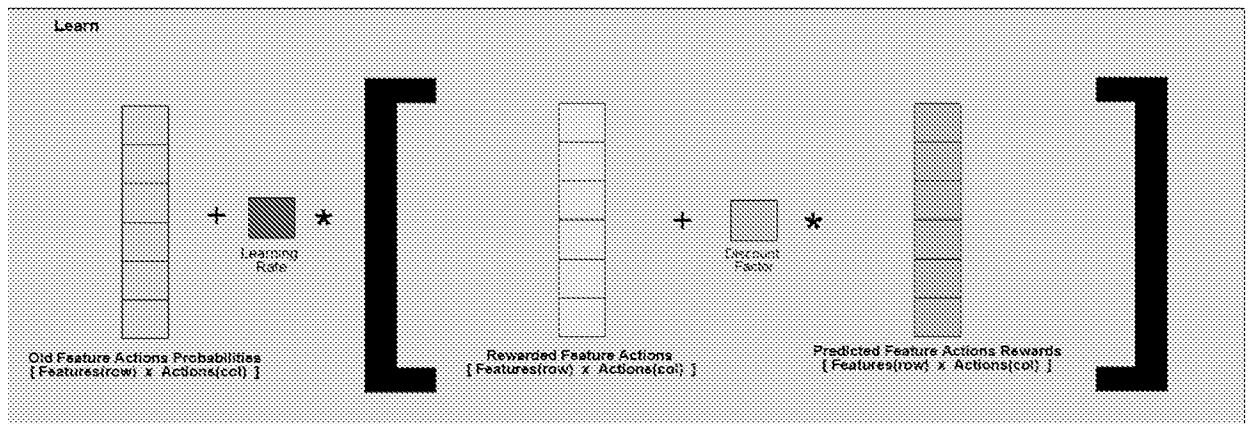


Figure 3: Q-Learn equation used to generate updated probabilities for feature policies.

- Reducing learning calculation time through selective updates.

One final advantage of our design is that by not loading all of the data from the database for computation, we also only need to update the feature action edge weights for the features that were loaded in section

Persisting policies in knowledge graphs. In doing so, we prevent the same issues for persisting data as for loading and can guarantee a faster load time that is based on only the active features in the feature vector.

Conclusion

Automating user interface quality assurance tests is hard, but not impossible. By applying a set of known algorithms in a very specific way, it is possible to reduce the number of paths that the system has to evaluate from millions to thousands. When coupled with elastic feature vectors, the number of actions to be evaluated on a path to elicit a state change is further reduced by ensuring that the most likely actions to produce a change are evaluated first. Along with the reduction in path set size, our modifications also provide a reduction in processing time needed for machine learning. This mixture of algorithms, heuristics, and machine learning improvements means that the system is capable of identifying and building user interface quality assurance tests in a shorter time period. In computer science, this improvement would be defined as a reduction of complexity of an NP-Hard problem such that its solution can be computed in provable time period as is expected of simpler NP (non-nondeterministic polynomial time) problems.

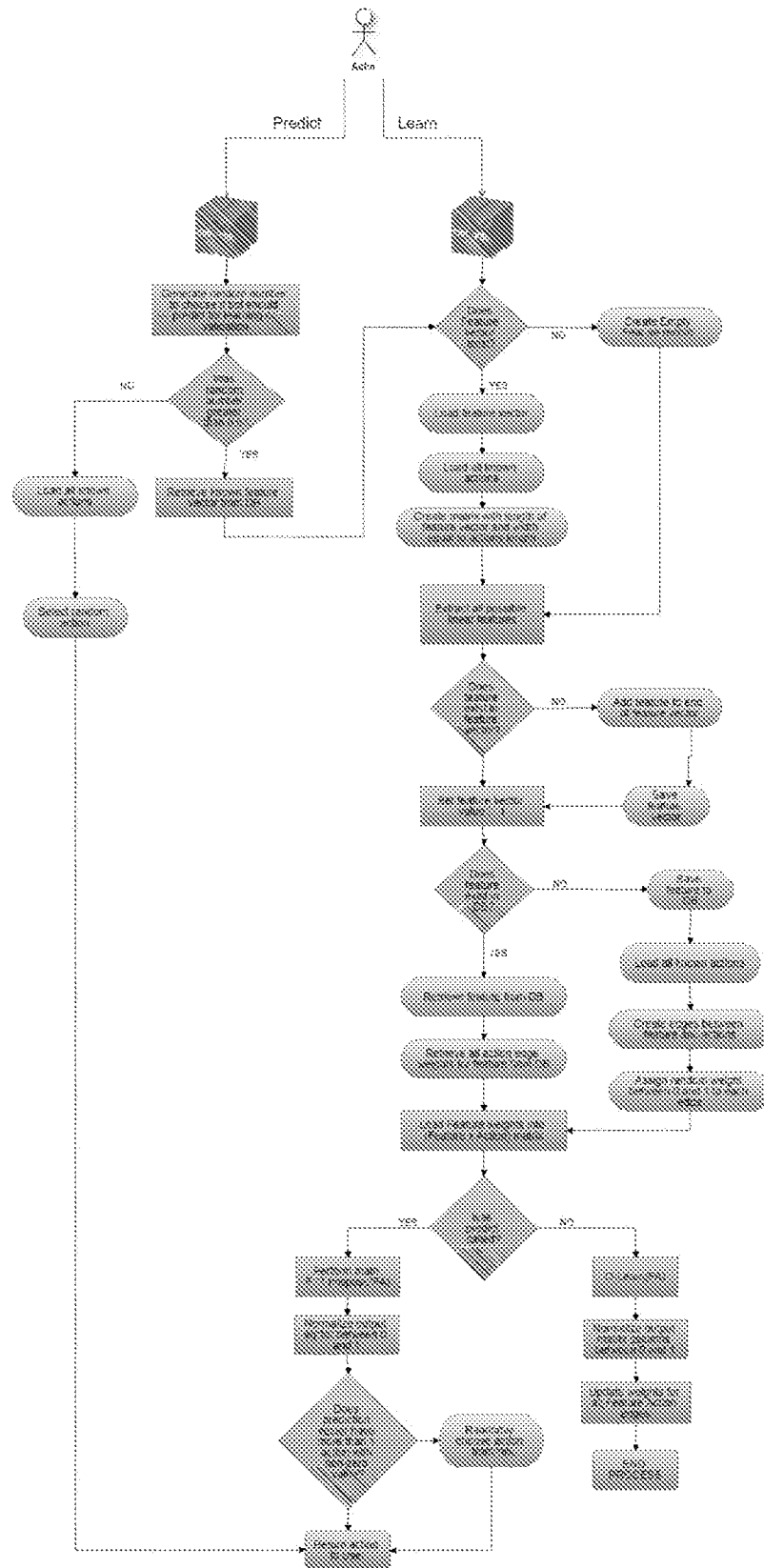


Figure 4: Flow chart describing the process the system follows for predicting and learning

Claims

1. A method of conducting quality assurance for a web site or web application having a plurality of interactive elements, the method comprising:
 - identifying, by a path expansion actor, a tree representing possible user interactions with the plurality of interactive elements;
 - predicting, by the path expansion actor based on an initial state of the web site or web application, a first predicted state of the web site or web application in response to a first action executed against a first element represented by a leaf node in the tree;
 - executing, by an exploratory actor, the first action against the first element;
 - observing, by the exploratory actor, a first state of the web site or web application in response to the first action;
 - performing a comparison of the first state to the first predicted state; and
 - refining a subsequent prediction by the path expansion actor based on the comparison of the first state to the first predicted state.
2. The method of claim 1, wherein predicting the first predicted state comprises operating on an elastic feature vector with an artificial neural network, the elastic feature vector representing the first action and the first element and the artificial neural network having weights representing a mapping between the first action and the first predicted state.
3. The method of claim 2, wherein refining the subsequent prediction comprises updating the weights of the artificial neural network based on the comparison of the first state to the first predicted state.
4. The method of claim 1, wherein executing the first action comprises emulating at least one of a mouse movement, mouse click, or keyboard stroke.
5. The method of claim 1, further comprising:
 - determining that the first predicted state matches the first state;
 - executing, by the exploratory actor, the first action against a second element represented by a subsequent node in the tree;
 - observing, by the exploratory actor, a second state of the web site or web application in response to executing the first action against the second element;
 - determining that the second state matches the first state; and

generating a user interface test based on the first element, the first action, and the first state.

6. The method of claim 5, further comprising:
updating the user interface test based on a change to the web site or web application.
7. The method of claim 1, further comprising:
determining that the first state matches the initial state; and
ceasing exploration of a corresponding branch of the tree by the exploratory actor.
8. The method of claim 1, further comprising:
identifying, by the path expansion actor, a second element in the plurality of interactive elements and a second action for executing against the second element; and
predicting, by the path expansion actor, a second change in state of the web site or web application in response to executing the second action against the second element.

Electronic Patent Application Fee Transmittal

Application Number:

Filing Date:

Title of Invention:

Automated Quality Assurance for Web Sites and Web Applications

First Named Inventor/Applicant Name:

Brandon Kindred

Filer:

Christopher M. Colice/Carol Perry

Attorney Docket Number:

QANA-001/00US 333031-2001

Filed as Small Entity

Filing Fees for Provisional

Description

Fee Code

Quantity

Amount

**Sub-Total in
USD(\$)**

Basic Filing:

PROVISIONAL APPLICATION FILING FEE

2005

1

140

140

Pages:

Claims:

Miscellaneous-Filing:

Petition:

Patent-Appeals-and-Interference:

Post-Allowance-and-Post-Issuance:

Description	Fee Code	Quantity	Amount	Sub-Total in USD(\$)
Extension-of-Time:				
Miscellaneous:				
Total in USD (\$)				140

Electronic Acknowledgement Receipt

EFS ID:	32778475
Application Number:	62679097
International Application Number:	
Confirmation Number:	7342
Title of Invention:	Automated Quality Assurance for Web Sites and Web Applications
First Named Inventor/Applicant Name:	Brandon Kindred
Customer Number:	58249
Filer:	Christopher M. Colice/Carol Perry
Filer Authorized By:	Christopher M. Colice
Attorney Docket Number:	QANA-001/00US 333031-2001
Receipt Date:	01-JUN-2018
Filing Date:	
Time Stamp:	11:32:13
Application Type:	Provisional

Payment information:

Submitted with Payment	yes
Payment Type	DA
Payment was successfully received in RAM	\$ 140
RAM confirmation Number	060118INTEFSW00009021501283
Deposit Account	501283
Authorized User	Carol Perry

The Director of the USPTO is hereby authorized to charge indicated fees and credit any overpayment as follows:

37 CFR 1.17 (Patent application and reexamination processing fees)

37 CFR 1.19 (Document supply fees)

File Listing:

Document Number	Document Description	File Name	File Size(Bytes)/ Message Digest	Multi Part /.zip	Pages (if appl.)
1	Provisional Cover Sheet (SB16)	QANA00100US_ProvCS.pdf	86384	no	3
			823953c8321db445682470ca34c2d70ae7657560		

Warnings:

This is not a USPTO supplied Provisional Cover Sheet SB16 form.

Information:

2	Specification	QANA00100US_ProvAppl.pdf	1584197	no	21
			9fdc0bacd9b1b2e4412bb03b468b27ba65d75fea		

Warnings:**Information:**

3	Fee Worksheet (SB06)	fee-info.pdf	29922	no	2
			0a5ba3f62e3409b879936fe7229372f11ada947c		

Warnings:**Information:**

Total Files Size (in bytes):			1700503
-------------------------------------	--	--	---------

This Acknowledgement Receipt evidences receipt on the noted date by the USPTO of the indicated documents, characterized by the applicant, and including page counts, where applicable. It serves as evidence of receipt similar to a Post Card, as described in MPEP 503.

New Applications Under 35 U.S.C. 111

If a new application is being filed and the application includes the necessary components for a filing date (see 37 CFR 1.53(b)-(d) and MPEP 506), a Filing Receipt (37 CFR 1.54) will be issued in due course and the date shown on this Acknowledgement Receipt will establish the filing date of the application.

National Stage of an International Application under 35 U.S.C. 371

If a timely submission to enter the national stage of an international application is compliant with the conditions of 35 U.S.C. 371 and other applicable requirements a Form PCT/DO/EO/903 indicating acceptance of the application as a national stage submission under 35 U.S.C. 371 will be issued in addition to the Filing Receipt, in due course.

New International Application Filed with the USPTO as a Receiving Office

If a new international application is being filed and the international application includes the necessary components for an international filing date (see PCT Article 11 and MPEP 1810), a Notification of the International Application Number and of the International Filing Date (Form PCT/RO/105) will be issued in due course, subject to prescriptions concerning national security, and the date shown on this Acknowledgement Receipt will establish the international filing date of the application.